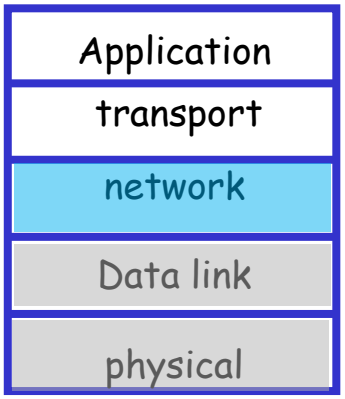
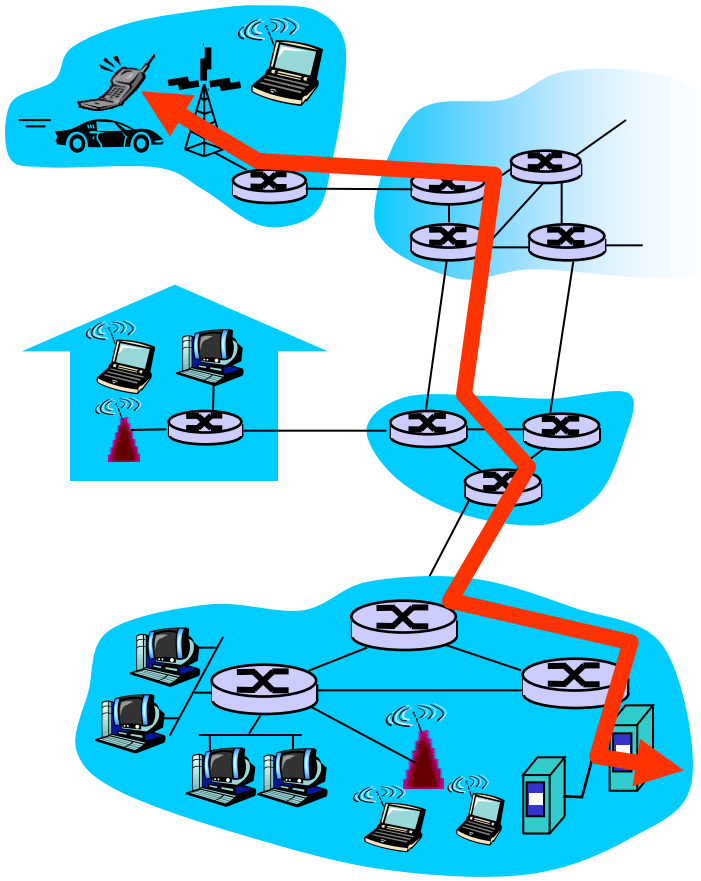
ENEL 573 – Computer Networks

Chapter 4: Network Layer

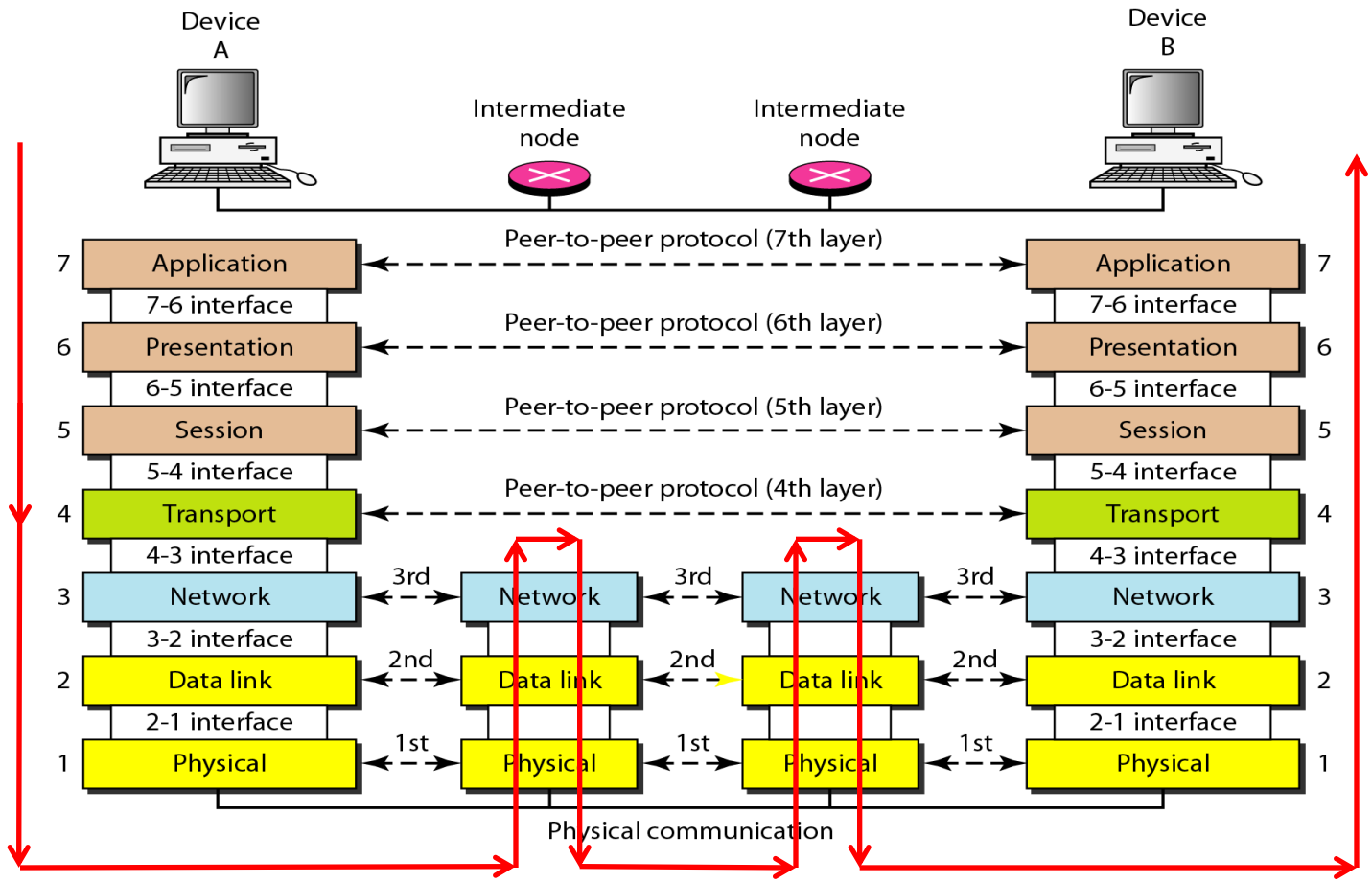
Instructor: Dr. Qiang (John) Ye
Department of ESE
University of Calgary, Canada
Email: qiang.ye@ucalgary.ca

- Transport layer provides process-to-process delivery
- Network Layer – supervises host-to-host packet delivery. Hosts could be separated by several physical networks
- Data-link layer provides node-to-node (hop-to-hop) delivery





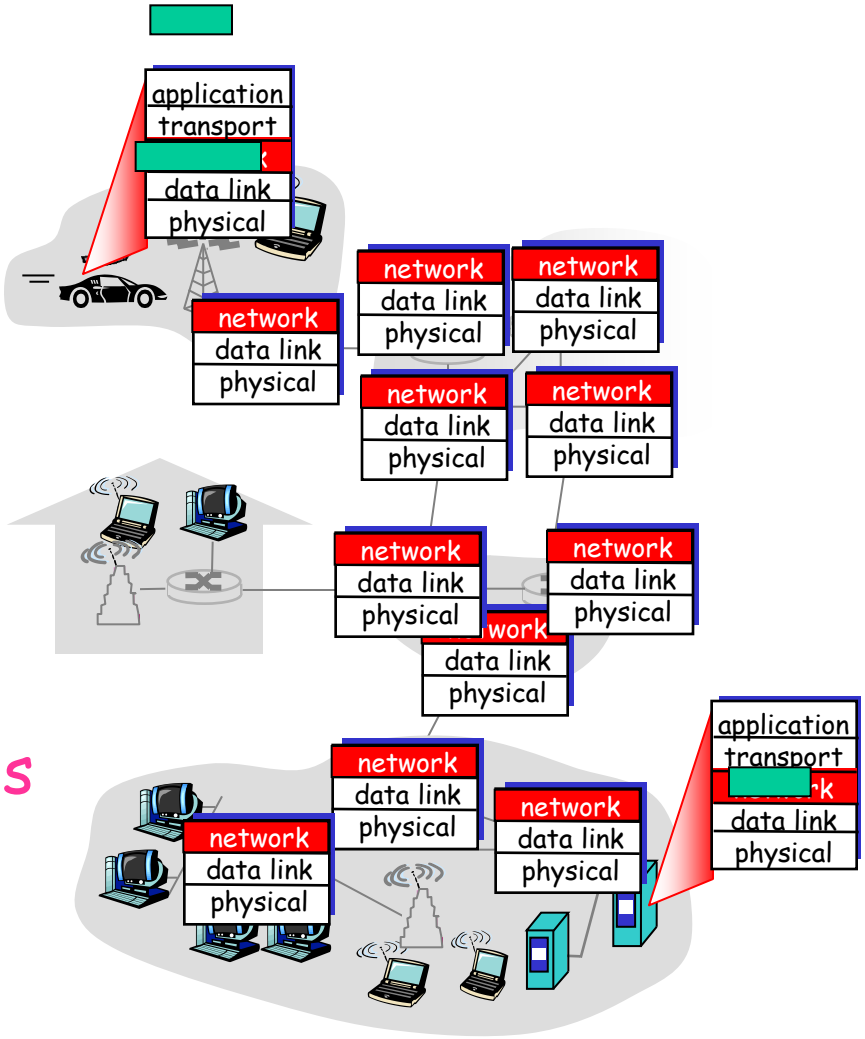
Introduction

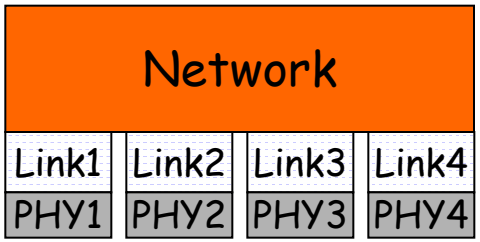


- Network layer protocols are implemented in every host and router.

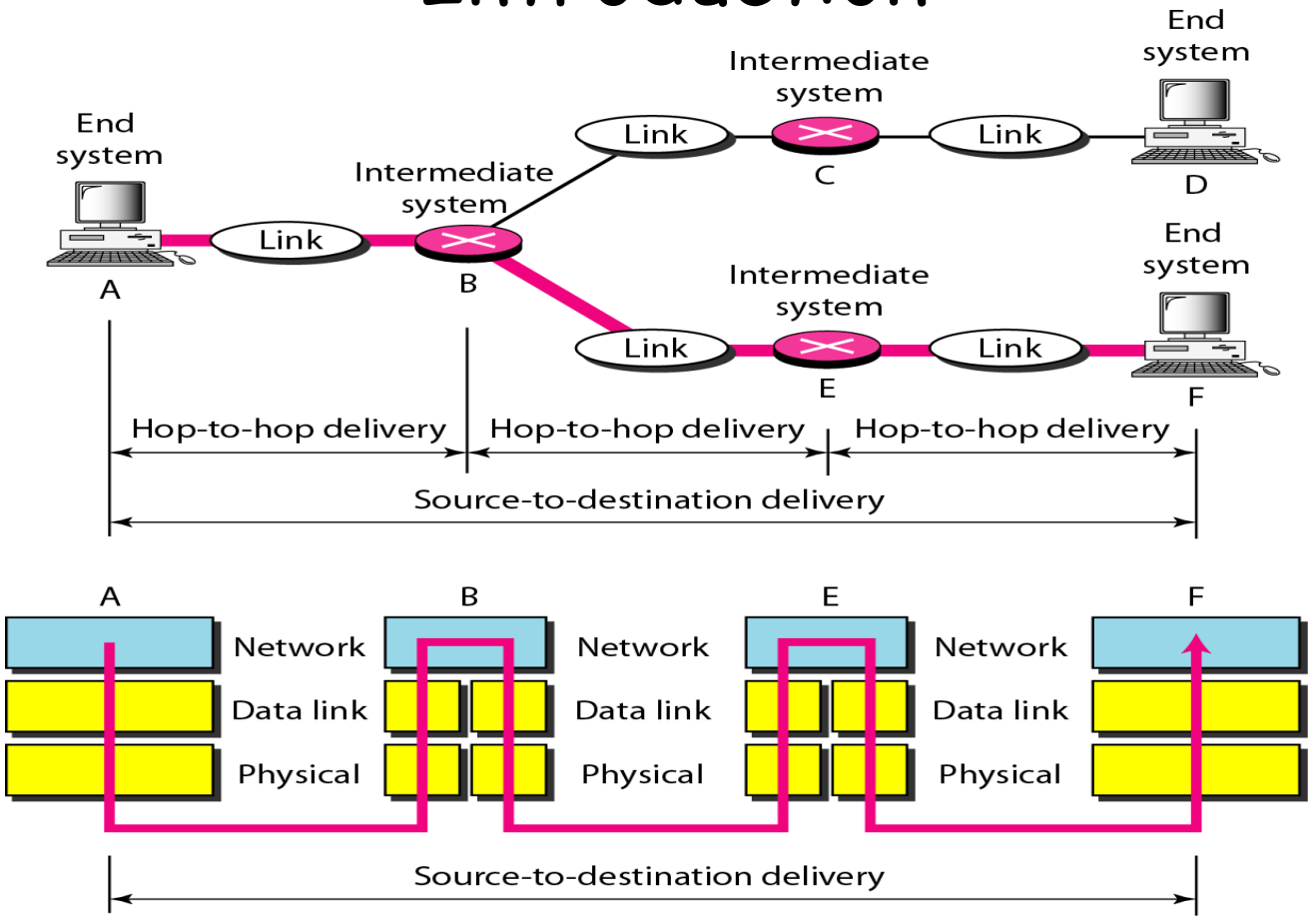
Network layer

- transport segment from sending to receiving host
- on sending side **encapsulate** segments into datagrams
- on rcving side, deliver segments to transport layer
- network layer protocols in *every* host, router
- router **examines** header fields in all IP datagrams passing through it



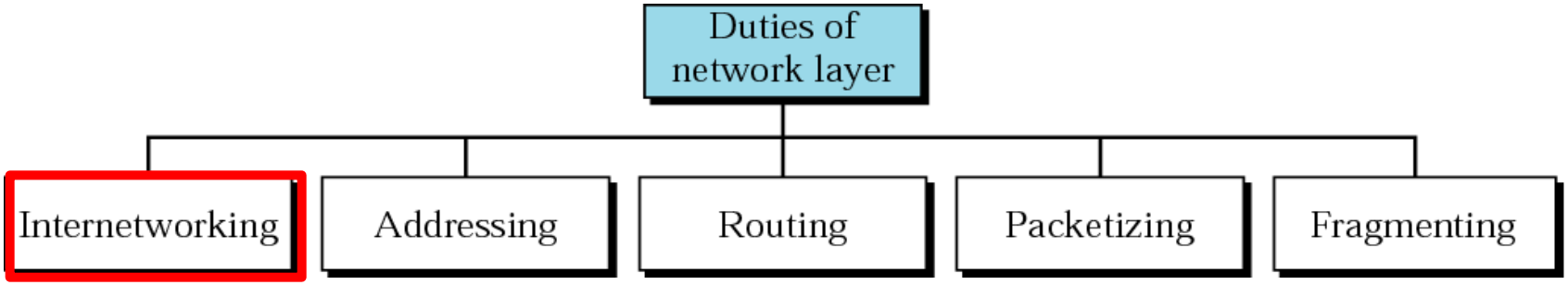


Introduction



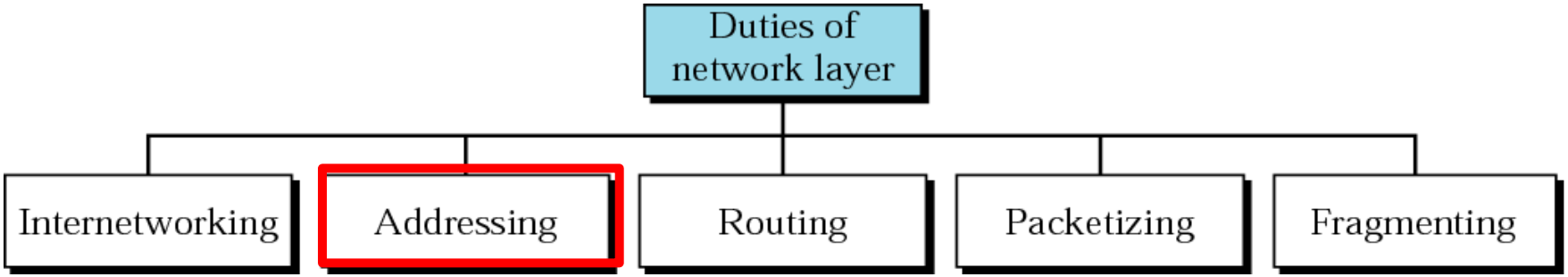
- Data link layer is responsible for delivery data from a node to a physically adjacent node in the same network.
- Network layer is responsible for delivering data between end hosts probably in different networks.

Network layer duties



- Internetworking: interconnecting different networks (various LAN technologies, telephone network, satellite link, ATM networks etc.) and making them look the same to the upper layer
- Hides underlying technologies details of physical networks from end user and application programs
- Allow transparent interconnection across different network technologies on a global scale
- Able to move datagrams between any hosts on the internetwork

Network layer duties

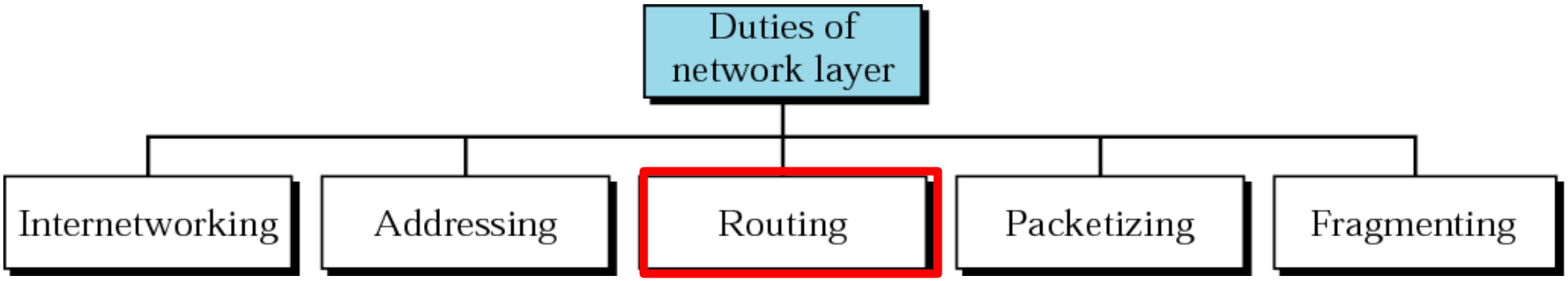


- How do we identify hosts on a global scale?

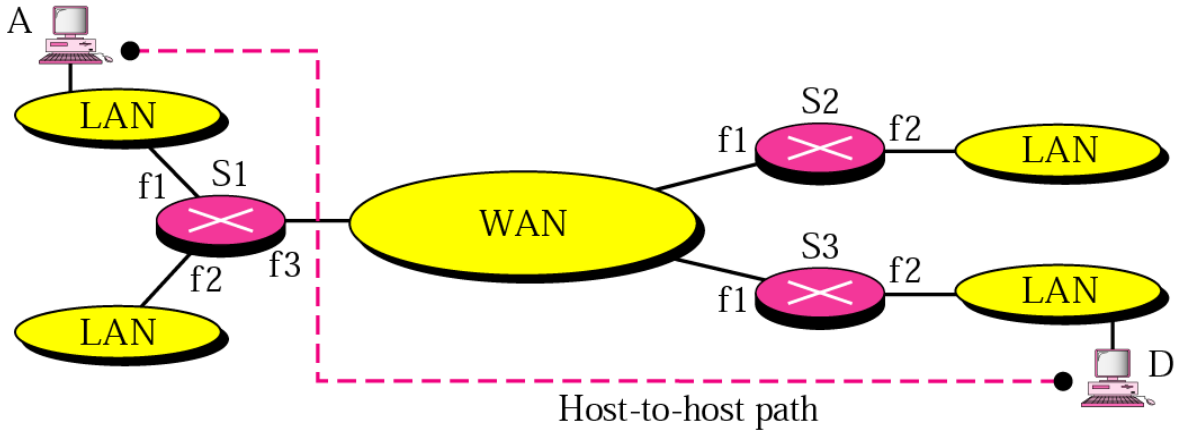
- The addresses must be uniquely and universally defined (host/router/machine/device/user) to the internet.

- Two devices on the internet can never have the same address. (Address per connection to the network)

Network layer duties

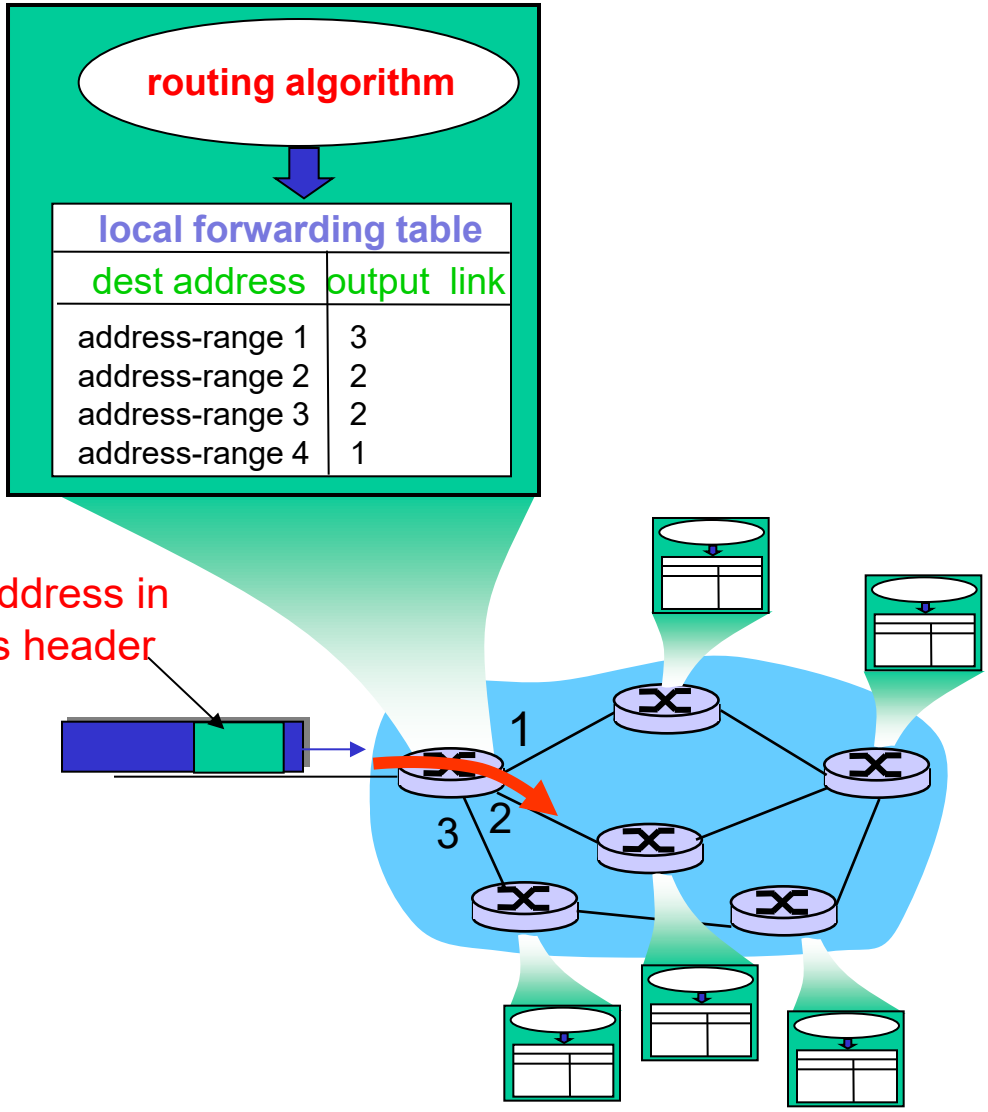


- Each packet reaches its destination via several routes.
- Which route is suitable or optimum?

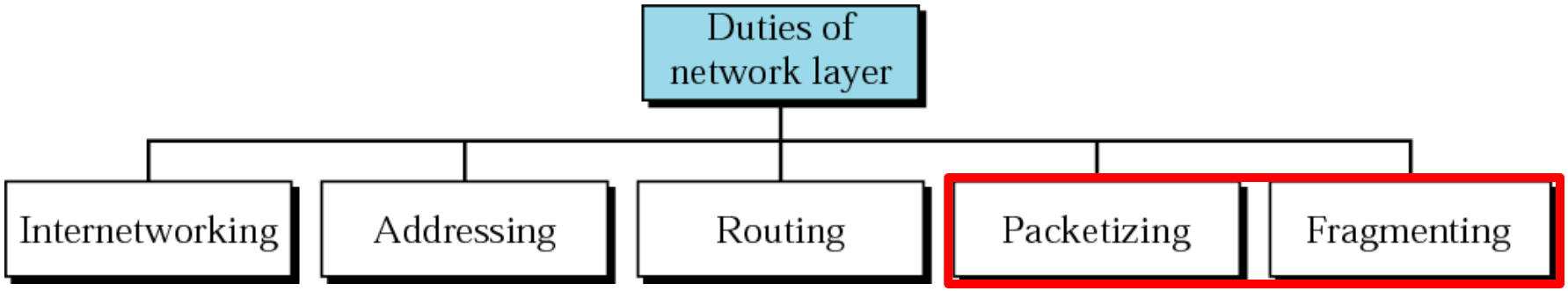


To deliver a datagram from A to D, how S1 knows which interface datagram should be forwarded to

Datagram Forwarding table



Network layer duties



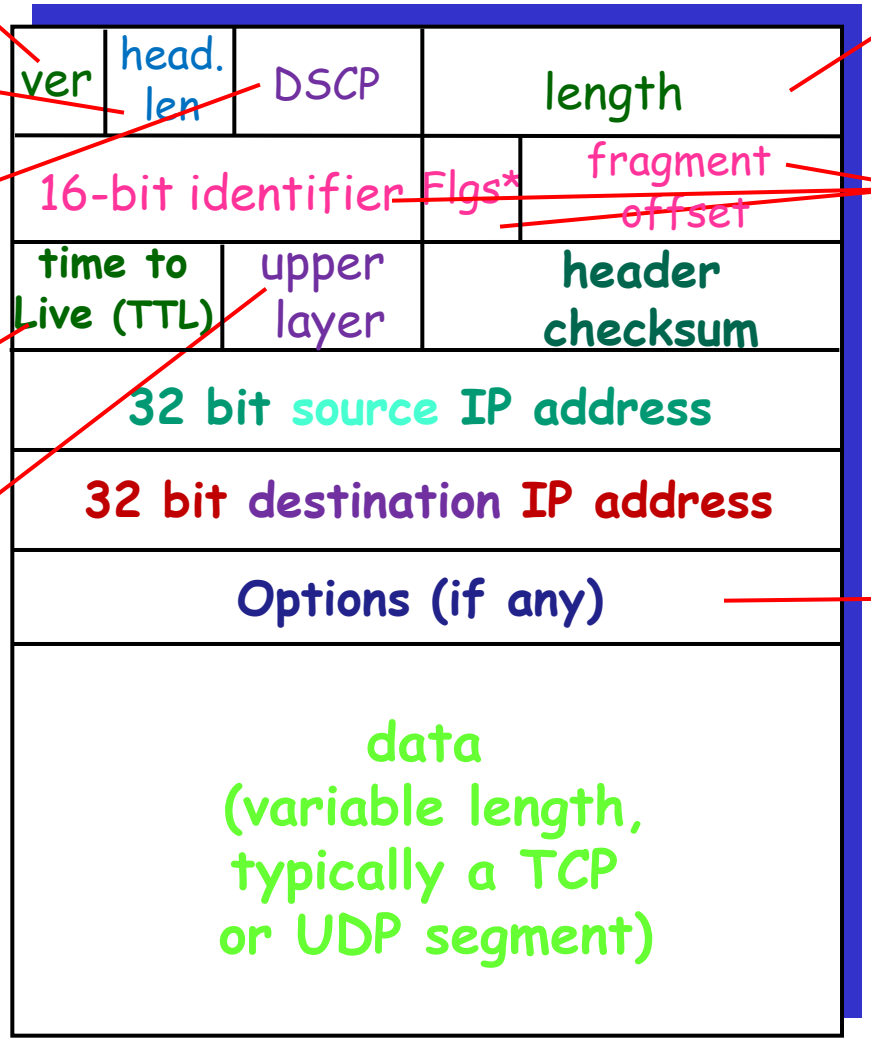
Packetizing:

- At sender: Network layer encapsulates segments received from upper layer protocols into datagram
- At receiver: decapsulate datagrams and deliver to transport layer.

Fragmenting/reassembly:

- A datagram can travel through **different networks** with different **maximum frame size**.
- The network layer must be able to **fragment datagrams into smaller units** so that they can be transferred over **various data-link layer technologies**.

IP Packet Format



IP protocol version number

header length (x4 bytes)
Typically has 20 byte header

Differentiated Services Code Point: "type" of data

max number of remaining hops (decremented at each router)

upper layer protocol to deliver payload to
Ex.: TCP = 6.

total datagram length (bytes)

for fragmentation/ Reassembly

E.g. timestamp, record route taken, specify list of routers to visit.

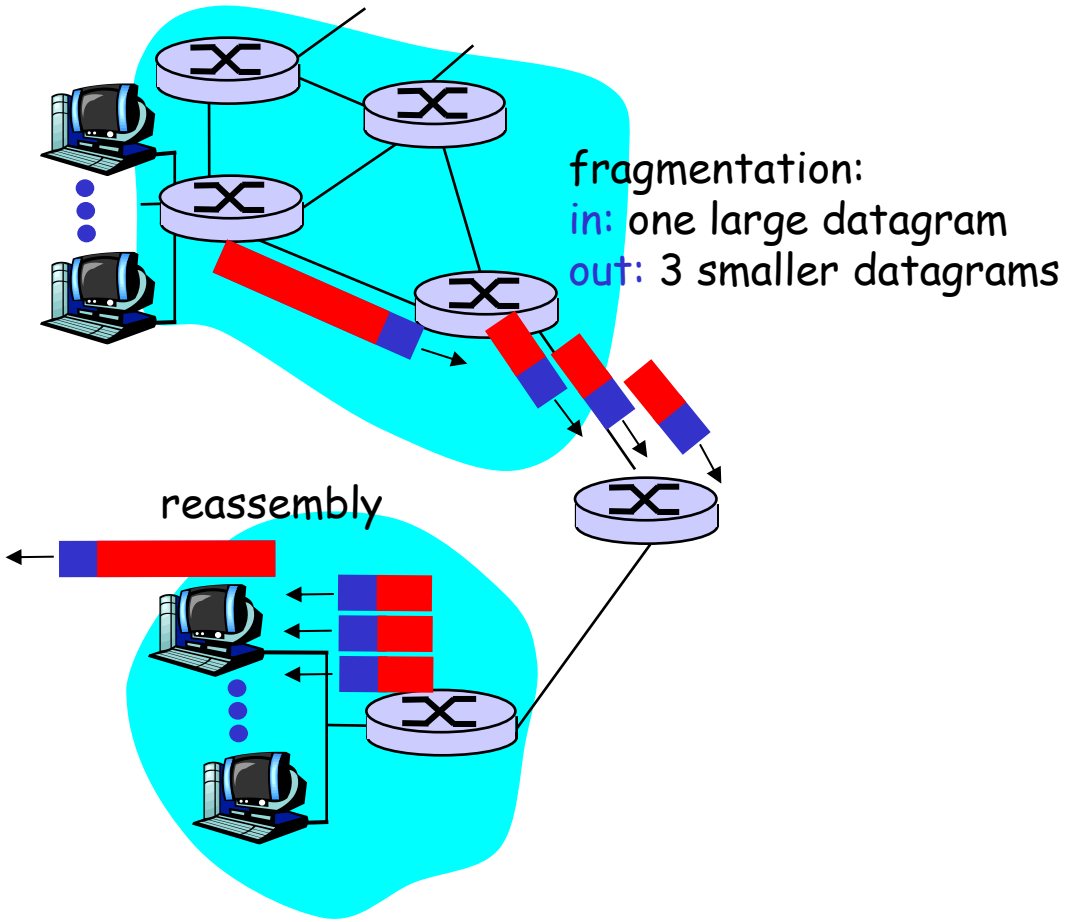
how much overhead with TCP?

- **20 bytes of TCP**
- **20 bytes of IP** (40 for IPv6)
- **= 40 bytes** + app layer overhead

*3-bit Flags: <Not used, Don't fragment, More fragments to follow,>

IP Fragmentation and Reassembly

- network links have MTU (max. transmission unit) - largest possible link-level frame.
  - different link types, different MTUs
- large IP datagram divided ("fragmented") within net
  - one datagram becomes several datagrams
  - "reassembled" only at final destination
  - IP header bits used to identify, order related fragments



IP Fragmentation and Reassembly

Identification: 16-bit field

- uniquely identifies datagram originating from source host
- increments by one for each datagram
- identification field is copied into all fragments
- identification helps destination in reassembling fragmented datagrams
- fragments with same identification from the same source should be assembled into one datagram

Flags: 3-bit field

- 1st bit is reserved
- 2nd bit is called "do not fragment" bit
  = 1, machine must NOT fragment datagram
- 3rd bit = 1, datagram is not the last fragment – there are more fragments after this one
  = 0, this is last or only fragment

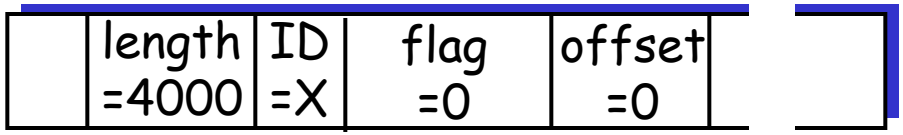
Offset: 13-bit field

- shows relative position of fragment's data in whole datagram
- the offset is measured in units of 8 bytes

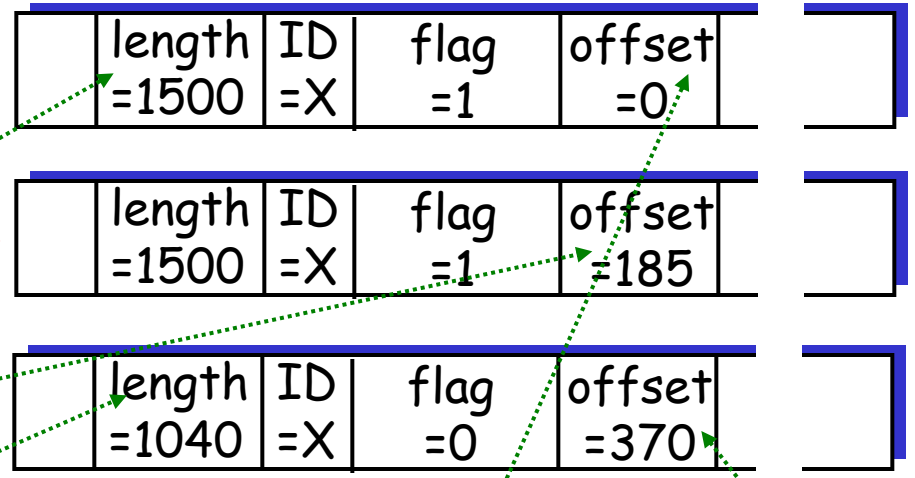
IP Fragmentation and Reassembly

Example

- 4000-byte datagram
- 3980 data bytes
- MTU = 1500 bytes
- Offset value is specified in units of 8-byte chunks, reflecting the position the data should be inserted



One large datagram becomes several smaller datagrams



1480 bytes in data field

offset =1480/8

3980-2*1480=1020 and 1020+20=1040
Should be inserted at byte 1480
Note 185*8=1480

The data should be inserted beginning at byte 0

185+185=370

Network layer service models

Network Architecture	Service Model	Guarantees ?				Congestion feedback
		Bandwidth	Loss	Order	Timing	
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR	constant rate	yes	yes	yes	no congestion
ATM	VBR	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR	guaranteed minimum	no	yes	no	yes
ATM	UBR	none	no	yes	no	no

- Network layer protocols: IP (Internet protocol)
- The internet provides best effort service → not guarantees on bandwidth, delay, ..

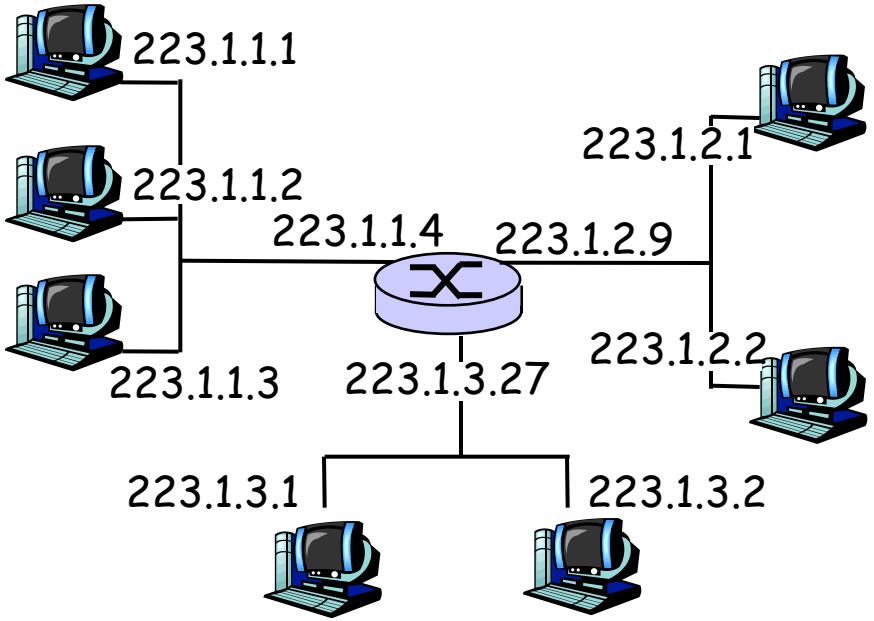
IP Addresses

- The address in the network layer of the TCP/IP model is called Internet Address or IP address

- IPv4 address: 32-bit (4 byte) identifier that uniquely and universally defines a connection between a host or a router and the network.

- Router's typically have multiple interfaces (multiple IPs)

- A host may have multiple interfaces (e.g., wired Ethernet, wireless 802.11)

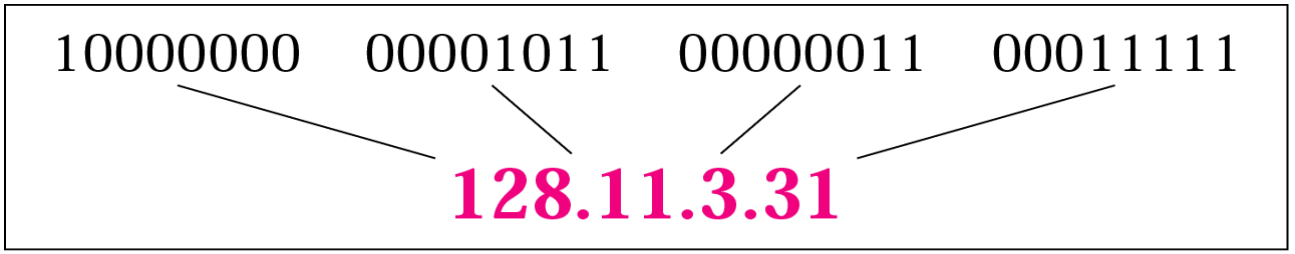


IP Addresses

- The IP addresses are unique (each connection has a different address) and universal (must be accepted by any host wants to connect to the internet).

- IP format: X.Y.W.Z (four 8-bit numbers)

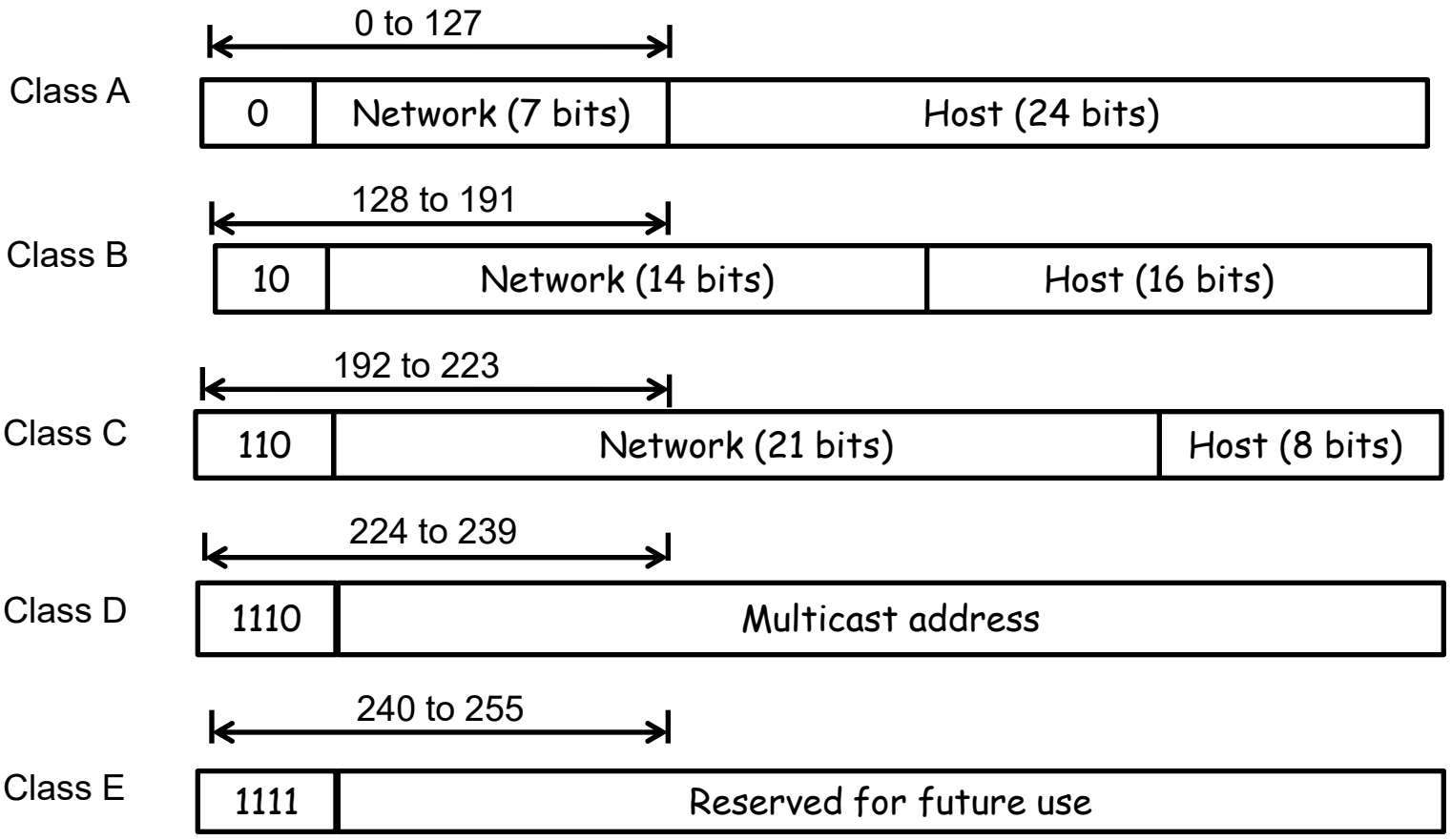
- Four decimal values, each number represents eight bits to denote each number between 0 and 255)



10000001 00001011 00001011 11101111 = 129.11.11.239

The address space of IPv4 is 2^{32} or more than 4 billion addresses.

Classful Addressing



- IP addresses are divided into five classes: A, B, C, D, and E.
- **Initial bits determine class**
- **Class D & E**: Reserved for special purposes and not available to organizations.

Classful Addressing

1. Given the address 23.56.7.91, find the network address.

   The class is A. Only the first byte defines the netid. We can find the network address by replacing the hostid bytes (56.7.91) with 0s. Therefore, the network address is 23.0.0.0.

2. Given the address 132.6.17.85, find the network address.

   The class is B. The first 2 bytes defines the netid. We can find the network address by replacing the hostid bytes (17.85) with 0s. Therefore, the network address is 132.6.0.0.

3. Given the network address 220.34.76.0, find the class, the block, and the range of the addresses.

   The class is C because the first byte is between 192 and 223. The block has a netid of 220.34.76. The addresses range from 220.34.76.0 to 220.34.76.255.

Classful Addressing

- IP addresses are hierarchical: consists of network id and host id
- routers only need to know the network portion – not the address of each individual host. The routing tables do not contain individual IPs but network addresses
- Classes A, B, and C are available to organizations
- Class A: Only 2^7 (128) networks can belong to this class - each has 2^{24} addresses (> 16million)
- Class B: 2^{14} (16384) networks -Each has 2^{16} (65536) addresses.
- Class C: 2^{21} networks, each has 2^8 (256) addresses
- Classful Routing protocols: RIP version1, IGRP

Classful Addressing

- Class A networks: too big, too few.

- Class C networks: too small, too many.

- Class B networks: not enough (run out already).

- In classful addressing, the network address is the one that is assigned to the organization.

- The number of addresses in class A and B may be larger than the needs of organizations.

- Millions of class A addresses are wasted.

- Many of class B addresses are wasted.

- The number of addresses in class C is smaller than the needs of most organizations.

Classful addressing problems

- IP addresses are quickly running out

- Bad utilization of addresses: Total number of addresses is 4 billion - should have enough addresses for everybody.

- The fact that the addresses are assigned in sets (or groups) significantly restricts the number of usable addresses.

  Class A and B are too large for most organizations . Many

  classes A and B addresses are wasted -- Class C are too small

- IP routing tables are growing very large

  Growth in number of networks in Internet reflected in number of table entries

Classful addressing problems

Short-term solution:

How to have an addressing scheme that meets exactly the needs of the users ?

- Subnetting
- Supernetting

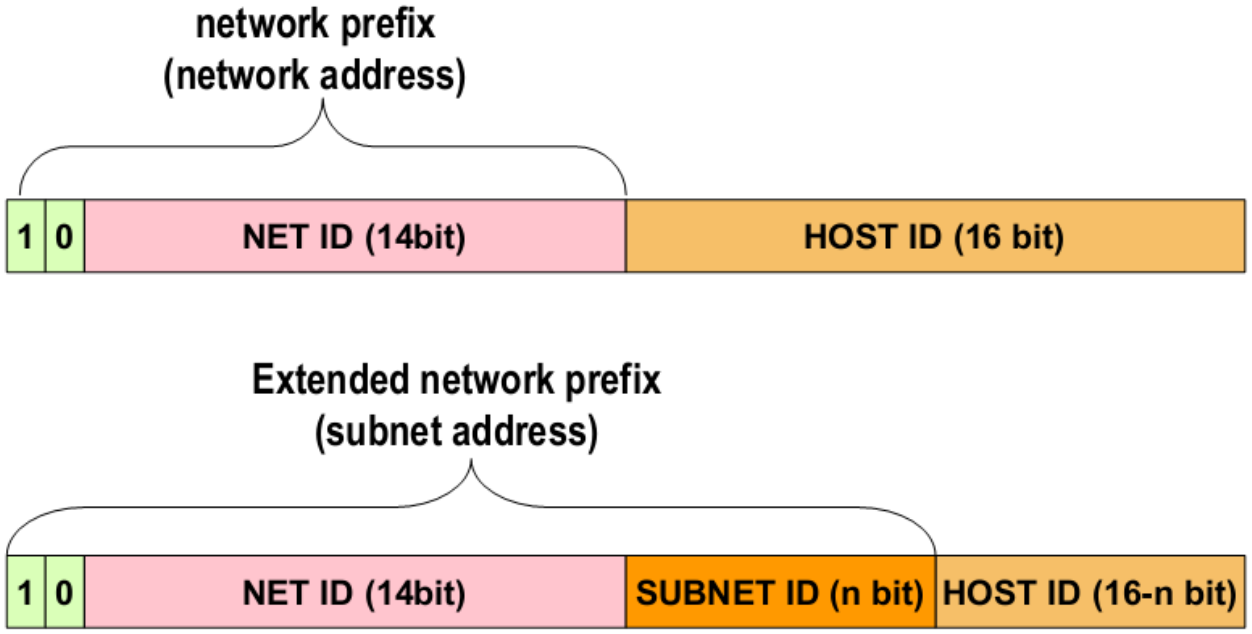
Long-term solution:

IPv6 with much bigger address space. The IPv4 address shortage was one of the reasons behind the IPv6.

Subnetting

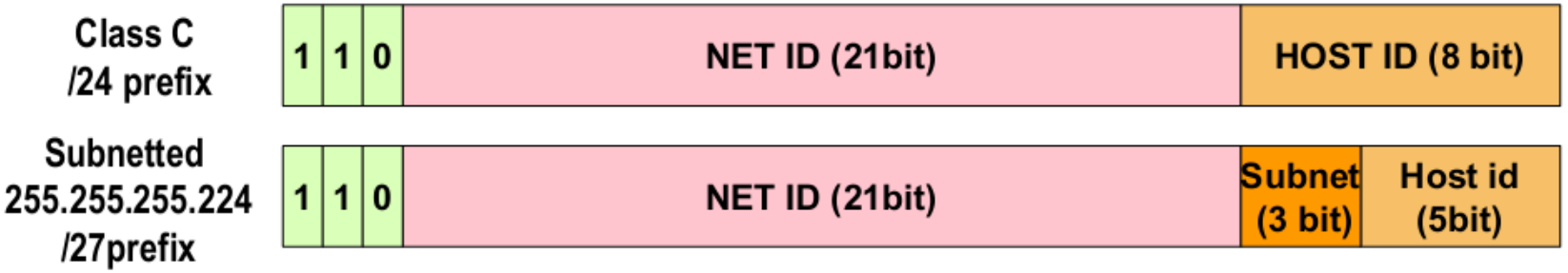
- In Subnetting: A network is divided into several smaller groups (subnets) with each group having its own subnet IP address but still appear as a single network to the rest of the Internet.
- IP address will have 3 levels (main network, subnet, host)
- Host portion of address partitioned into subnet number and host number

Class B address example



Subnetting

Base net 11000001.00000001.00000001.00000000 193.1.1.0/24

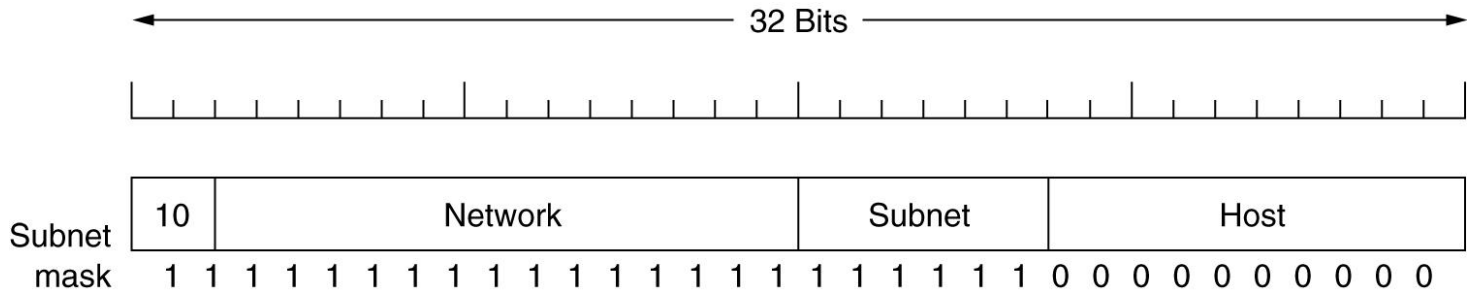


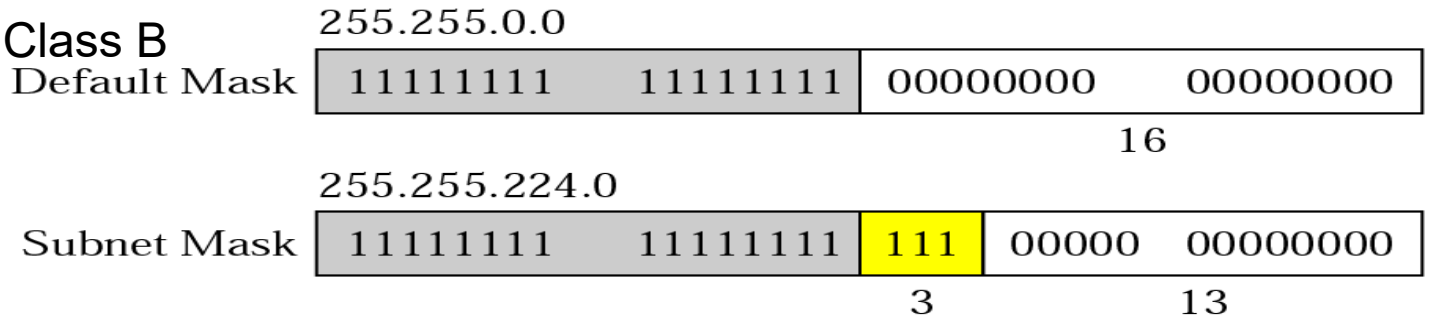
Subnet # 0	11000001.00000001.00000001.**000**00000	**193.1.1.0/27**
Subnet # 1	11000001.00000001.00000001.**001**00000	**193.1.1.32/27**
Subnet # 2	11000001.00000001.00000001.**010**00000	**193.1.1.64/27**
Subnet # 3	11000001.00000001.00000001.**011**00000	**193.1.1.96/27**
Subnet # 4	11000001.00000001.00000001.**100**00000	**193.1.1.128/27**
Subnet # 5	11000001.00000001.00000001.**101**00000	**193.1.1.160/27**
Subnet # 6	11000001.00000001.00000001.**110**00000	**193.1.1.192/27**
Subnet # 7	11000001.00000001.00000001.**111**00000	**193.1.1.224/27**

Remember: maximum 30(2^5-2) hosts attachable to each subnet

Subnet mask

- Subnet mask is a 32-bit consists of zeros and ones
- Subnet masks specify where the split between network, subnet and host portions is
- Default/subnet mask when ANDed with the IP address it gives the network/subnet addresses





Subnet mask

1. In original IP: IP has two portions network address and host address

To get network address → host part set to 0 → use the class default mask

Classes default masks

Class	In Binary	In Dotted-Decimal	Using Slash
A	11111111 00000000 00000000 00000000	255.0.0.0	/8
B	11111111 11111111 00000000 00000000	255.255.0.0	/16
C	11111111 11111111 11111111 00000000	255.255.255.0	/24

2. In subnetted IP: IP has three portions network address, subnet address, and host address

To get subnet address → host part set to 0 → use the subnet mask

Network/subnet addresses: identify the network/subnet to which the host is connected

Subnet Address and Mask

- Host IP address:
  159.100.9.18 — 10011111.01100100.00001001.00010010
- Class B –network/default mask:
  255.255.0.0 — 11111111.11111111.00000000.00000000

Default/subnet mask bitwise ANDed with the IP address gives the network/subnet addresses

Network addr: 159.100.0.0 — 10011111.01100100.00000000.00000000

- Subnet Mask
  Longer than natural class mask;
  Example: class B address with 5 bits subnet id
  Subnet mask = /21 (/prefix-length notation) — 11111111.11111111.11111000.00000000
  Subnet mask = 255.255.248.0 (dot decimal notation)

Subnet addr: 159.100.8.0 — 10011111.01100100.00001000.00000000

To avoid ambiguity: 159.100.8.0/21

Subnet Address and Mask

1- Subnet mask enables a computer to determine which computers are on the same subnet. This is very important for message routing.

E.g., IP address: 129.118.49.189

Subnet mask: 255.255.255.0

IP address: 129.118.49.x is for the computers in the same subnet

2- How does the router know which subnet a packet should go?

- External router has routing table based on network addresses.
- Internal router has routing table based on subnetwork addresses.

Subnetting

Organization has Class B address (16 host ID bits) with network address: 150.100.0.0. Create subnets with 100 addresses each

7 bits sufficient for each subnet, 16-7=9 bits for subnet ID

512 subnets each has 128 addresses

Range address for subnet X, where X is 9 bits

Starting address : 10010110 1100100 X 0000000

Start address = subnet address

Last address: 10010110 1100100 X 1111111

X takes values from 000000000 to 111111111

Note 10010110 = 150, 1100100 =100

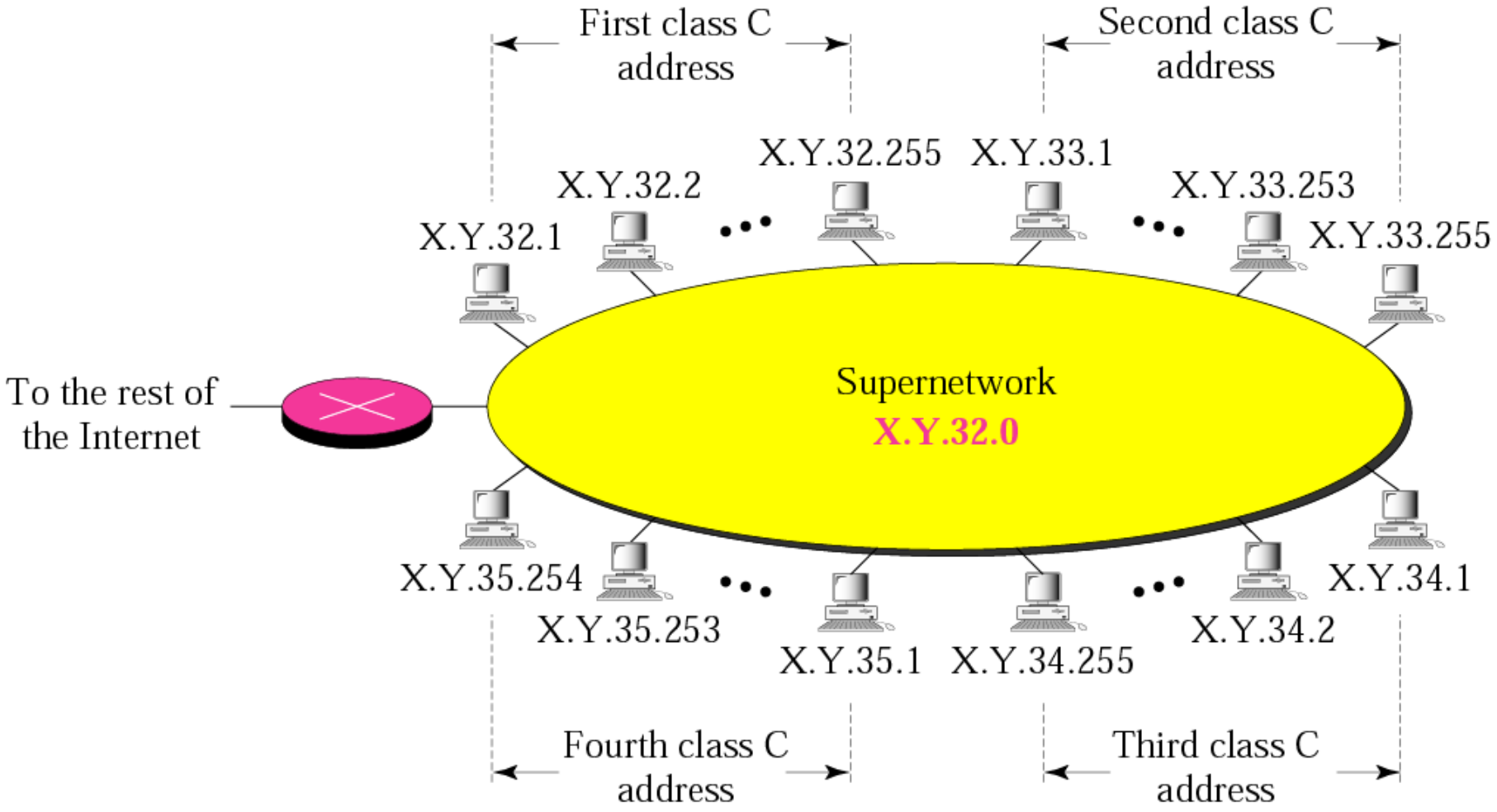
What is the Subnet mask?

11111111 11111111 11111111 10000000 or/25

Supernetting

- Class C has a lot of networks but the size of a block with a maximum number of 256 addresses may not satisfy the needs of an organisation.
- One solution is supernetting.
- In supernetting, an organisation can combine several class C blocks to create a supernetwork with a large range of addresses, but seen as one network by the rest of the internet.
- The number of blocks must be a power of 2
- The blocks must be contiguous in the address space (no gaps between the blocks).

Example of a supernetwork



Supernet Address

- 4 address-contiguous networks:

  213.2.96.0: 11010101. 00000010. 011000*00*. 00000000
  213.2.97.0: 11010101. 00000010. 011000*01*. 00000000
  213.2.98.0: 11010101. 00000010. 011000*10*. 00000000
  213.2.99.0: 11010101. 00000010. 011000*11*. 00000000

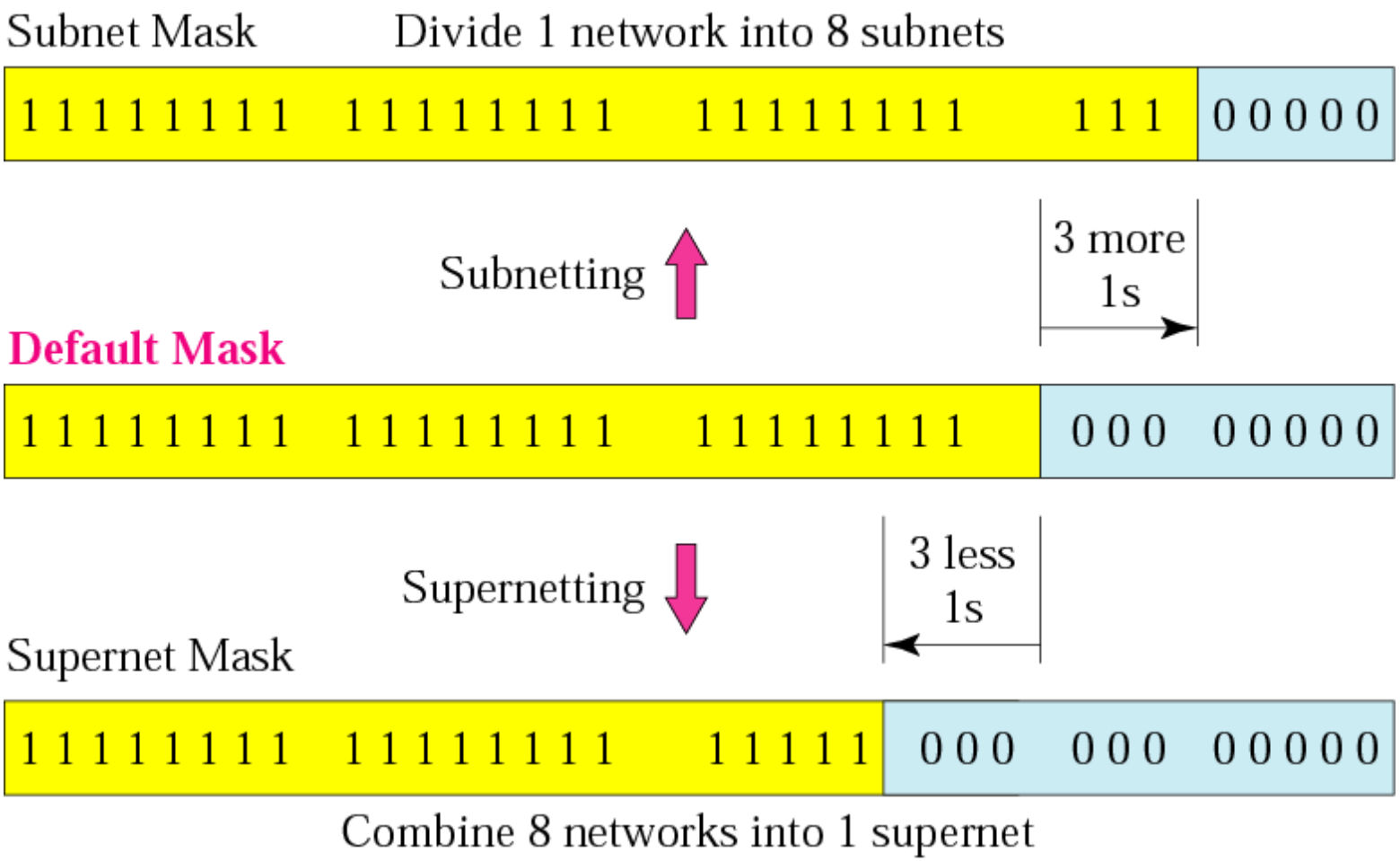
- Supernet mask:

  255.255.252.0

- Supernet address:

  213.2.96.0/22: 11010101. 00000010. 011000 00. 00000000

Comparison of subnet, default, and supernet masks



Example

A supernet has a first address of 205.16.32.0 and a supernet mask of 255.255.248.0. A router receives 3 packets with the following destination addresses:

205.16.37.44

205.16.42.56

205.17.33.76

Q: Which packet belongs to the supernet?

We apply the supernet mask to find the beginning address.

205.16.37.44 AND 255.255.248.0 ➔ 205.16.32.0

205.16.42.56 AND 255.255.248.0 ➔ 205.16.40.0

205.17.33.76 AND 255.255.248.0 ➔ 205.17.32.0

Only the first address belongs to this supernet.

Example 1:

- A small organization is given a block with the beginning address and the prefix length 205.16.37.24/29. What is the range of addresses in the block?
- The beginning address is 205.16.37.24. To find the last address we keep the first 29 bits and change the last 3 bits to 1s.

Beginning: **11001111 00010000 00100101 00011**000

Ending : **11001111 00010000 00100101 00011**111

There are only 8 addresses in this block.

- Alternatively, we can argue that the length of the suffix is 32 – 29=3. So there are 2^3 = 8 addresses in this block. If the first address is 205.16.37.24, the last address is 205.16.37.31 (24 + 7 = 31).

Example 2:

What is the network address if one of the addresses is 167.199.170.82/27?

Solution

The prefix length is 27, which means that we must keep the first 27 bits as it is and change the remaining bits (5) to 0s. The 5 bits affect only the last byte. The last byte is 01010010. Changing the last 5 bits to 0s, we get 01000000 or 64. The network address is 167.199.170.64/27.

And the address with the mask to get the base address of the subnet (the first address)

Example 3:

An organization is granted the network address block of 130.34.12.64/26. The organization needs to have four subnets. What are the subnet addresses and their range for each subnet?

The suffix length is 6 (32-26). This means the total number of addresses in the block is 64 (2^6). If we create four subnets, each subnet will have 16 addresses. Let us first find the subnet prefix (subnet mask). We need four subnets, which means we need to add two more '1's to the site prefix /26. The subnet prefix is then /28.

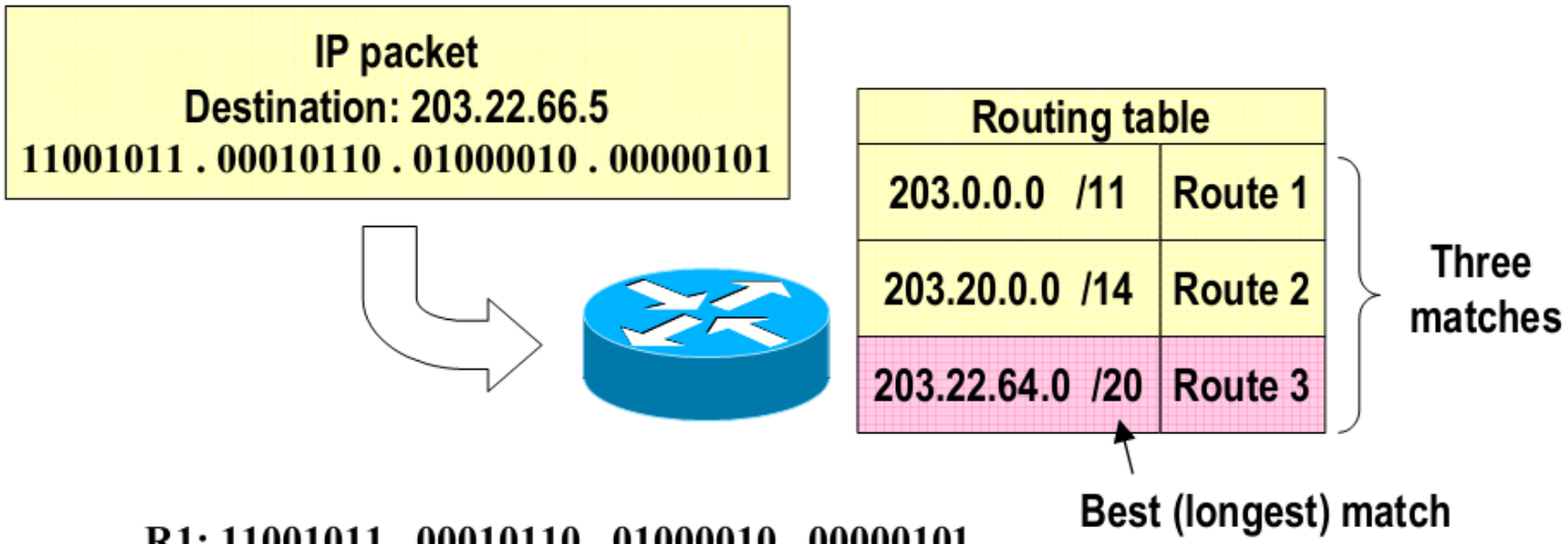
Subnet 1: 130.34.12.64/28 to 130.34.12.79/28.
Subnet 2 : 130.34.12.80/28 to 130.34.12.95/28.
Subnet 3: 130.34.12.96/28 to 130.34.12.111/28.
Subnet 4: 130.34.12.112/28 to 130.34.12.127/28.

Datagram Forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

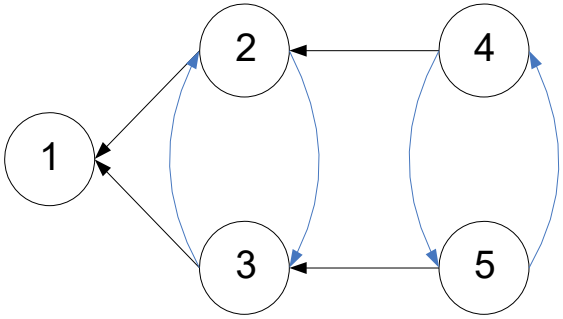
Longest prefix matching

when looking for forwarding table entry for given destination address, use longest address prefix that **matches** destination address.



Longest match (R3) = smaller network

Traffic Routing Algorithms



- The network layer guides the packets from their source to their destination, along a path that may comprise a number of links.

- The major function of network layer is: Routing – to find the best path for a particular transmission from the source to the destination. The job of routers is to forward packets from their source to their destination according to the Routing table.

- Routing algorithm usually includes a set of N nodes (routers) and a set of link lengths between these nodes: $\{d_{i,j},\ i, j \in S\}$, $d_{i,j}=\infty$ if there is no link from i to j.

- A router is a box with input and output links. It has access to the routing table, and contains software that enables it to determine which of the several possible paths between the source and the destination is the best for a particular transmission.
- "Good" routing algorithm:
  1. simplicity
  2. robustness to node/link failures
  3. stability (fast adaptation to topology and traffic changes)
  4. optimality (low delay and high throughput)
  5. fairness.
- Most of routing algorithms are distributed algorithms: routing decisions are made at each node in a cooperative manner. Information is exchanged with neighbors.
- Bellman-Ford algorithm is a distributed algorithm.

Initial condition:

At step **h=0**, node i has the estimate $d_i^h=\infty$, for all $i \in S$ with $i \neq D$ and that $d_D^h=0$ (D is destination). The nodes advertise these estimates to their neighbors.

At step h≥1:

Node j updates its estimate of its distance to D to some value d_j^h and advertises that value to its neighbors.

At step h+1:

Node i examines the message that it just received from its neighbors at the previous step h, e.g., j might be a neighbor of i, that is $d(i,j)<\infty$ and j informs i at step h that the shortest estimated distance from j to D is equal to d_j^h. Node i gets similar messages from all its neighbors. Node i then know that if it chooses to send a packet to D by first sending it to its neighbor j, then the distance that the packet will travel to reach D is estimated to be $d(i,j)+ d_j^h$. Since node i can choose the neighbor to which it sends the message destined to D, node i estimates the shortest distance to D as $d_i^{h+1} =\min\{d(i,j)+ d_j^h, j \in S\}$.

The Bellman-Ford Algorithm

1.The shortest path length from node i to the destination node is the sum of the length of the arc to the node following i and the shortest path length from that node to the destination node.

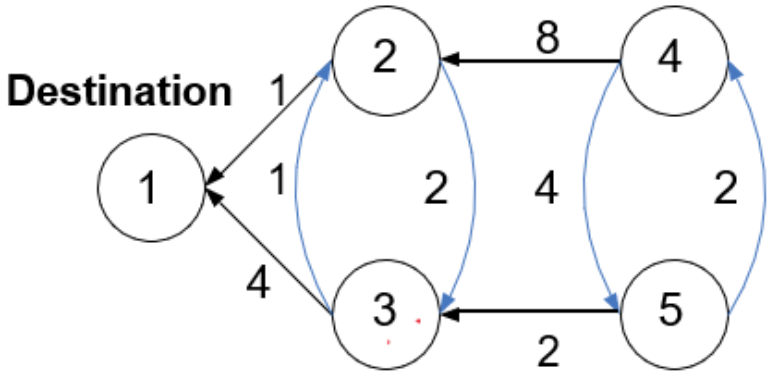
2.Algorithm:

$d_1^h = 0$ for all h

$d_i^{h+1} = \min_j [d(i,j) + d_j^h]$ for all $i \neq 1$

Starting from the initial conditions $d_i^0 = \infty$, for all $i \neq 1$.

Note: node 1 is the destination node. d_i^h is the distance from node i to node 1. h is the number of iterations. $d(i,j)$ is the distance from node i to node j.



Find the shortest paths from each of the other nodes to **destination node 1** on this digraph using the Bellman-Ford algorithm.

Example:

Given a digraph G(N,A) with the following specifications.

N={1,2,3,4,5,6}

A={(2,1)=1, (2,3)=2, (3,1)=5, (3,2)=1, (4,2)=6, (4,5)=2, (5,3)=2, (5,4)=3, (6,4)=1, (6,5)=2}

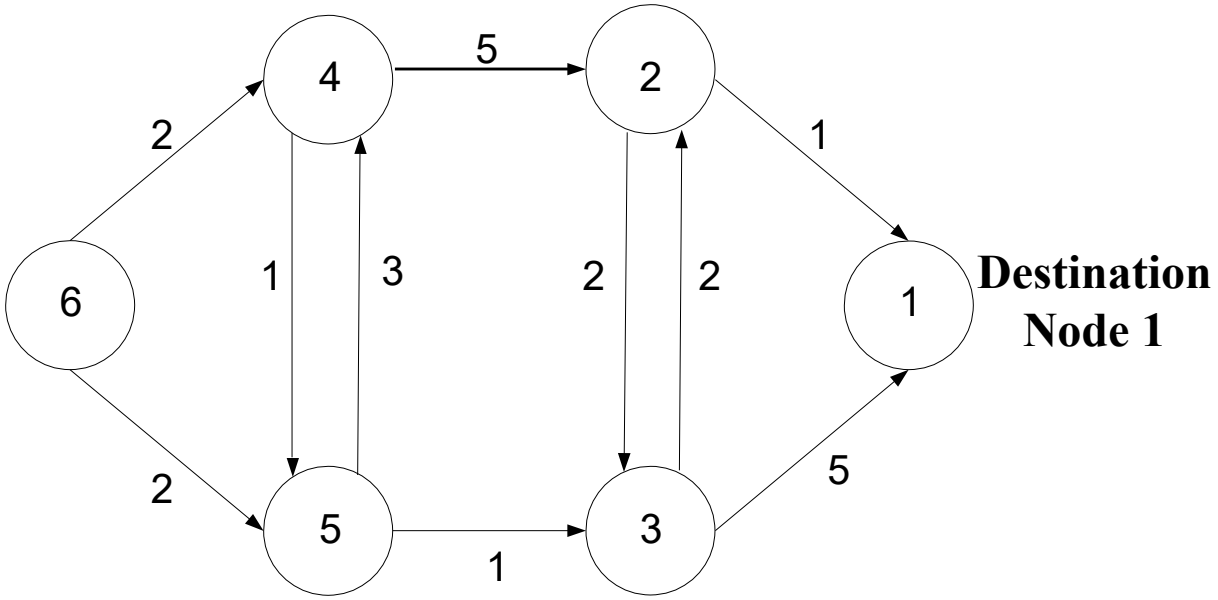
a) Draw and label the arcs of the digraph.

b) Discuss how the Bellman-Ford algorithm can be used to determine the shortest path from any node to a destination node on the digraph.

c) Find the shortest paths from each of the other nodes to **destination node 1** on this digraph using the Bellman-Ford algorithm.

Solution:

(a) The labeled digraph is as follows:



(b) The Bellman-Ford algorithm can be implemented on the above diagram.

1. Label each node i with (d_i, j) where d_i (the first element of the label) is **the distance from node i to the destination node** through the neighboring node j.
2. Initialization: label node i with (d_i^0, .) where

$$d_i^0 = \begin{cases} 0, & i = 1 \\ \infty, & i \neq 1 \end{cases}$$

3. Shortest distance labeling of all nodes: for each successive step h > 0, determine j which minimizes $d_i^h = d(i,j) + d_j^{h-1}$, and update label at each node i with (d_i^h,j).
4. Repeat step 3 until no change of any d_i occurs.

(c) The execution results of Bellman-Ford algorithm on the above digraph is shown in the following table

h	1	2	3	4	5	6
0	(0,.)	(∞, .)	(∞, .)	(∞, .)	(∞, .)	(∞, .)
1	(0,.)	(1,1)	(5,1)	(∞, .)	(∞, .)	(∞, .)
2	(0,.)	(1,1)	(2,2)	(7,2)	(7,3)	(∞, .)
3	(0,.)	(1,1)	(2,2)	(7,2)	(4,3)	(8,4)
4	(0,.)	(1,1)	(2,2)	(6,5)	(4,3)	(6,5)
5	(0,.)	(1,1)	(2,2)	(6,5)	(4,3)	(6,5)

From table, we can get all the shortest paths:

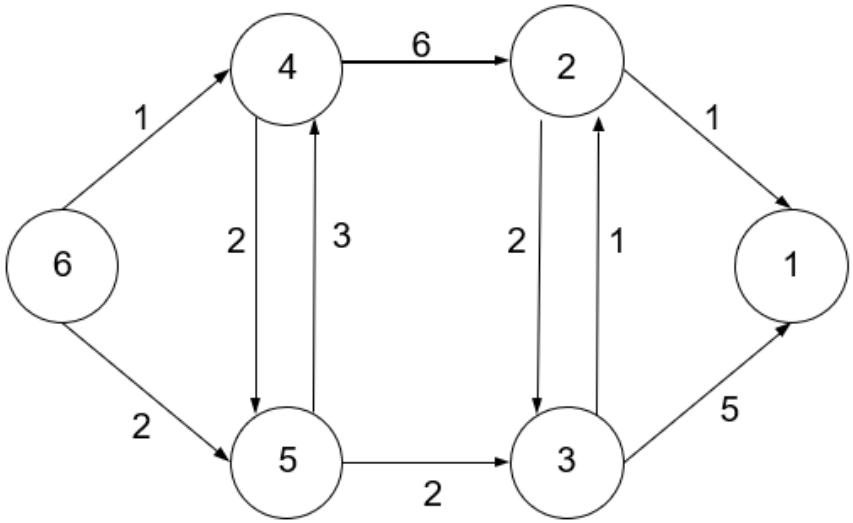
2 -> 1

3 -> 2 -> 1

4 -> 5 -> 3 -> 2 -> 1

5 -> 3 -> 2 -> 1

6 -> 5 -> 3 -> 2 -> 1



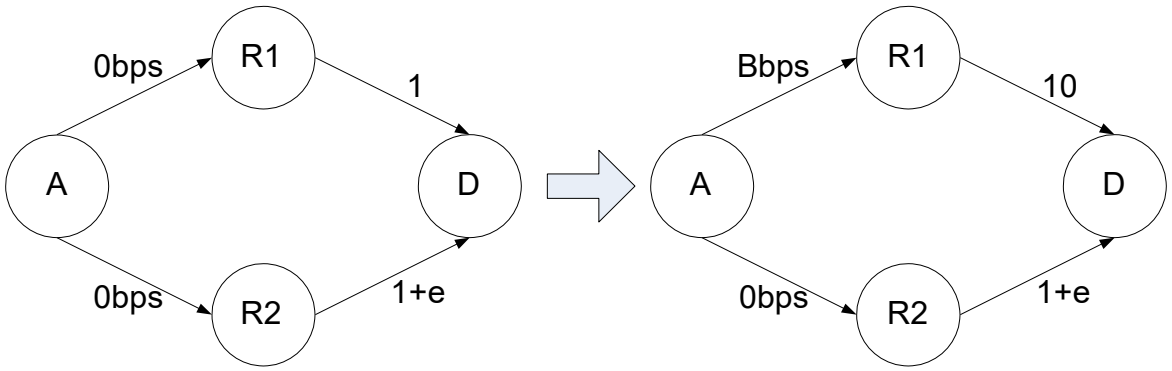
Advantage:

The algorithm is very flexible to the choice of initial estimates d_j^0. It can be executed at each node *i* in parallel with every other nodes.

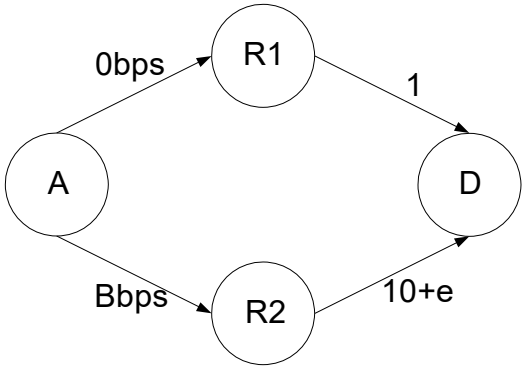
Disadvantage:

The routing uses only one path per original-destination pair, thereby potentially limiting the throughput; its capability to adapt to the changing traffic conditions is limited by its sensitivity to oscillations, which is due to the abrupt traffic shifts resulting from when some of the shortest paths change due to changes in link delay.

Example:



Source A is attached to two routers. Initially, A sends all its traffic to D via node R1. The delay along link (R1, D) is equal to 10 and delay along link (R2, D) is equal to 1+e. These routers advertise these delays to node A which then finds out that the preferred path to D starts with R2 and not with R1. Source A then sends all its traffic to R2 and not with R1.



As a result, the situation alternates between the two paths. A better routing strategy would consist in source A sending half of its traffic to R1 and the other half to R2.

Optimal routing can eliminate both of these disadvantages. It directs traffic exclusively along paths which are shortest with respect to some link delays that depend on the traffic flows carried by the links.

THANKS!